
ANALISI DELLA SICUREZZA NELLE COMUNICAZIONI TCP/IP

*Implementazione di un'Architettura Client/Server
con Simulazione di Attacchi Man-in-the-Middle in Python*

UNIVERSITÀ DEGLI STUDI DI URBINO
CARLO BO

Corso di Laurea in Informatica Applicata
Insegnamento di Reti di Calcolatori



Autore:

Stud. Giovanni Amadori [315477]

Docente:

Prof. Antonio Della Selva

Giugno 2026

Indice

1	Introduzione	1
1.1	Obiettivi	1
1.2	Processo di Sviluppo	1
2	Architettura e Specifiche Tecniche	2
2.1	Architettura Generale del Sistema	2
2.1.1	Modulo Client	2
2.1.2	Modulo Server	3
2.1.3	Modulo Man-in-the-Middle	4
2.2	Sistema di Cifratura e Decifratura	4
2.3	Tecnologie e Librerie Utilizzate	5
2.3.1	Libreria <code>socket</code>	5
2.3.2	Libreria <code>threading</code>	5
2.3.3	Modulo <code>crypto</code>	5
2.4	Gestione degli errori e casi particolari	6
2.5	Test	6
2.5.1	Test Comunicazione in Chiaro	7
2.5.2	Test Comunicazione Cifrata	8
2.5.3	Test Gestione Errori	9
3	Considerazioni Finali	11
3.1	Risultati Ottenuti	11
3.2	Limiti del Sistema Implementato	11
3.3	Possibili Miglioramenti Futuri	12
3.4	Considerazioni Personali	12
	Bibliografia	13

1 Introduzione

In un panorama tecnologico in cui la trasmissione dei dati rappresenta la spina dorsale di qualsiasi infrastruttura digitale, la sicurezza delle comunicazioni in rete è diventata una priorità assoluta.

Questa relazione è dedicata all'analisi delle vulnerabilità del protocollo TCP/IP attraverso lo sviluppo di un'architettura Client/Server e la successiva simulazione di un attacco informatico di tipo Man-in-the-Middle (MitM), implementato in linguaggio Python.

L'obiettivo fondamentale dell'elaborato è dimostrare la fragilità delle comunicazioni che viaggiano “in chiaro” e analizzare l'efficacia delle contromisure crittografiche per la protezione dei dati.

La scelta di questo argomento nasce dal personale desiderio di comprendere le dinamiche della Cybersecurity, approfondendo non solo gli aspetti teorici ma anche quelli pratici legati alla sicurezza delle reti.

I recenti scenari di conflitto globale dimostrano come la cyberwarfare sia ormai utilizzata per il furto di informazioni strategiche e la compromissione dei sistemi informatici.

Partendo da questa considerazione, lo sviluppo di un applicativo in grado di intercettare il traffico di rete ha permesso di simulare il punto di vista di un potenziale attaccante, con l'obiettivo di comprendere meglio le tecniche di difesa dei dati sensibili.

Il progetto è stato sviluppato esclusivamente in linguaggio Python utilizzando le librerie standard, senza l'impiego di dipendenze esterne complesse.

Al fine di realizzare la simulazione, l'intera architettura è stata ospitata in ambiente locale sul computer personale. Questa scelta ha permesso di operare in un contesto controllato e isolato, ideale per testare scenari di intercettazione del traffico di rete in condizioni di sicurezza.

1.1 Obiettivi

Gli obiettivi principali del progetto sono i seguenti:

- Simulare una comunicazione tra entità Client e Server su rete TCP/IP in ambiente locale.
- Studiare il comportamento della trasmissione dati in presenza di un attacco di tipo Man-in-the-Middle.
- Valutare le differenze tra trasmissione in chiaro e trasmissione protetta mediante cifratura dei messaggi.

1.2 Processo di Sviluppo

Il processo di sviluppo del progetto si è articolato in tre fasi principali:

- **Progettazione dell'architettura di rete:** definizione del modello Client/Server e realizzazione degli script in Python, con l'obiettivo di stabilire una connessione TCP e gestire lo scambio bidirezionale dei messaggi.
- **Simulazione della minaccia Man-in-the-Middle:** introduzione di un nodo intermedio incaricato di intercettare il traffico tra Client e Server, evidenziando le vulnerabilità delle comunicazioni non protette.
- **Implementazione dei meccanismi di sicurezza:** integrazione di un sistema di cifratura e decifratura basato su crittografia asimmetrica RSA e crittografia simmetrica AES, realizzato a livello applicativo su Client e Server, con l'obiettivo di analizzare l'effetto della protezione dei dati sulla visibilità del traffico da parte del nodo Man-in-the-Middle.

2 Architettura e Specifiche Tecniche

In questa sezione vengono descritte le specifiche tecniche e l'architettura del sistema sviluppato. Il sistema è composto da tre componenti principali: il modulo Client, il modulo Server e un nodo intermedio Man-in-the-Middle (MitM), tutti implementati in linguaggio Python.

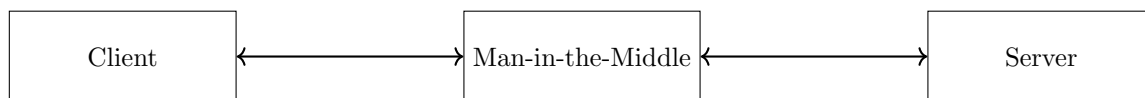
L'obiettivo dell'architettura è simulare una comunicazione su rete TCP/IP e analizzare il comportamento del flusso di dati in differenti condizioni di sicurezza, confrontando la trasmissione in chiaro con una trasmissione protetta mediante un algoritmo di cifratura e decifratura sviluppato ad hoc.

2.1 Architettura Generale del Sistema

L'architettura del sistema implementato è di tipo Client/Server, ovvero un modello di progettazione in cui la comunicazione avviene tra due entità principali che interagiscono attraverso una connessione di rete basata su socket TCP:

- **Client:** si occupa di avviare la comunicazione, inviare messaggi verso il Server e gestire l'interazione lato utente.
- **Server:** riceve i messaggi in ingresso, li elabora e fornisce una risposta al Client, svolgendo il ruolo di nodo centrale di gestione dei dati.

All'interno dell'architettura è inoltre presente un nodo intermedio di tipo Man-in-the-Middle (MitM), che si inserisce nel canale di comunicazione tra Client e Server. Questo componente ha la funzione di intercettare il traffico dati in transito, consentendo l'analisi dei messaggi scambiati e la simulazione di uno scenario di attacco in rete.



Il sistema è stato progettato per operare in due modalità distinte:

- **Modalità in chiaro:** in cui i dati vengono trasmessi senza alcuna forma di protezione e risultano quindi direttamente leggibili dal nodo intermedio.
- **Modalità cifrata:** in cui i messaggi vengono protetti tramite un algoritmo di cifratura e decifratura sviluppato appositamente, con l'obiettivo di garantire la riservatezza delle informazioni durante il trasferimento.

2.1.1 Modulo Client

Il modulo Client rappresenta l'entità che avvia la comunicazione all'interno dell'architettura sviluppata. La sua funzione principale è quella di instaurare una connessione con il Server attraverso il protocollo TCP/IP e consentire lo scambio bidirezionale di messaggi.

L'implementazione è stata realizzata in linguaggio Python mediante l'utilizzo della libreria standard socket, la quale permette al Client di creare connessioni di rete e scambiare dati con il Server tramite TCP/IP.

Il comportamento del modulo Client durante l'esecuzione può essere suddiviso nelle seguenti fasi operative:

1. **Inizializzazione della connessione:** il Client crea una socket TCP e tenta di stabilire una connessione con l'endpoint configurato.
2. **Fase di handshake crittografico:** il Client attende la ricezione della chiave pubblica RSA dal Server. Una volta ricevuta, genera una chiave AES, la cifra con la chiave ricevuta dal server e la invia.
3. **Attivazione della comunicazione dati:** l'utente inserisce i messaggi che vengono cifrati mediante l'algoritmo AES prima della trasmissione.
4. **Ricezione della risposta:** il Client riceve la risposta del Server e la decifra utilizzando la chiave AES condivisa.
5. **Terminazione della connessione:** la connessione termina su richiesta dell'utente o alla chiusura del collegamento.

Dal punto di vista della sicurezza, il Client opera senza essere a conoscenza della presenza del nodo intermedio nella comunicazione, considerando il Server come destinatario diretto dei messaggi inviati e mantenendo il comportamento previsto in una normale comunicazione tra endpoint.

2.1.2 Modulo Server

Il modulo Server rappresenta l'entità che gestisce la comunicazione all'interno dell'architettura sviluppata, ricevendo e rispondendo alle richieste provenienti dal Client. La sua funzione principale è quella di mantenere attivo il canale di comunicazione e garantire lo scambio di messaggi all'interno della connessione stabilita.

L'implementazione è stata realizzata in linguaggio Python mediante l'utilizzo della libreria standard socket, che consente la creazione di un endpoint in ascolto su una porta specifica.

Il funzionamento del modulo Server può essere descritto attraverso una sequenza di fasi operative principali:

1. **Inizializzazione del servizio di ascolto:** il Server crea una socket TCP e si mette in attesa di connessioni su un endpoint configurato.
2. **Gestione della connessione in ingresso:** una volta ricevuta una richiesta, il Server accetta la connessione e instaura il canale comunicativo con il Client.
3. **Fase di handshake crittografico:** il Server genera una coppia di chiavi RSA e invia la chiave pubblica al Client. Quest'ultimo genera una chiave simmetrica AES, la cifra mediante la chiave pubblica ricevuta e la invia al Server, che la decifra utilizzando la propria chiave privata RSA.
4. **Ricezione ed elaborazione dei messaggi:** il Server riceve i messaggi inviati dal Client, li decifra utilizzando la chiave AES condivisa ed esegue l'elaborazione prevista.
5. **Invio della risposta:** il Server restituisce al Client una risposta di conferma cifrata, completando il ciclo di comunicazione.
6. **Terminazione della connessione:** la comunicazione viene interrotta su richiesta del Client o in caso di chiusura del collegamento.

Dal punto di vista della sicurezza, il Server opera senza consapevolezza della presenza del nodo intermedio nel canale di comunicazione, trattando il Client come unico interlocutore diretto.

2.1.3 Modulo Man-in-the-Middle

Il modulo Man-in-the-Middle (MitM) rappresenta il nodo intermedio inserito nella comunicazione tra Client e Server, con il compito di intercettare il traffico dati scambiato tra le due entità.

L'implementazione è stata realizzata in linguaggio Python mediante l'utilizzo della libreria standard `socket`, che consente di creare un punto di ascolto e di gestire lo scambio dei messaggi tra i due endpoint della comunicazione. Inoltre, per permettere l'esecuzione concorrente delle diverse operazioni di ricezione e inoltro dei dati, è stata utilizzata la libreria standard `threading`, impiegata per la gestione dei flussi di esecuzione paralleli.

All'avvio del sistema, il nodo MitM si interpone tra Client e Server, agendo come intermediario trasparente rispetto alle due entità comunicanti. In questo modo, sia il Client sia il Server continuano a operare come se la comunicazione fosse diretta, senza essere consapevoli della presenza del nodo intermedio.

Il comportamento del nodo MitM durante l'esecuzione può essere suddiviso nelle seguenti fasi operative:

1. **Intercettazione della connessione:** il MitM si pone in ascolto e intercetta la connessione stabilita dal Client verso il Server.
2. **Instradamento del traffico:** i dati ricevuti dal Client vengono inoltrati verso il Server e viceversa, mantenendo attiva la comunicazione. Durante questa fase viene inoltre introdotto un ritardo artificiale tramite la funzione `time.sleep()`, che simula la latenza dovuta all'intercettazione e all'elaborazione dei pacchetti da parte del nodo intermedio.
3. **Analisi dei dati transitanti:** durante il passaggio dei messaggi, il MitM può leggere il contenuto dei dati.

Il modulo evidenzia la vulnerabilità della comunicazione non protetta, in cui i dati scambiati risultano direttamente accessibili in caso di intercettazione. Nel caso di comunicazione cifrata, invece, il contenuto dei messaggi non è interpretabile senza la chiave di decifratura, limitando in modo significativo l'efficacia dell'intercettazione.

2.2 Sistema di Cifratura e Decifratura

Per garantire la riservatezza delle informazioni trasmesse tra Client e Server anche in presenza del nodo intermedio Man-in-the-Middle (MitM), all'interno del sistema è stato implementato un meccanismo di cifratura e decifratura basato sulla combinazione di crittografia asimmetrica RSA e crittografia simmetrica AES.

Il seguente sistema consente di sfruttare i vantaggi di entrambe le tecniche crittografiche. In particolare RSA, tramite una coppia di chiavi pubblica e privata, permette di condividere in modo sicuro una chiave senza trasmetterla in chiaro: la chiave pubblica viene utilizzata per cifrare le informazioni, mentre la corrispondente chiave privata consente di decifrare il contenuto originale. AES, invece, permette di cifrare e decifrare i dati con maggiore efficienza attraverso l'utilizzo di un'unica chiave condivisa tra le due entità.

Nello specifico, il processo di comunicazione protetta si articola nelle seguenti fasi:

1. **Generazione della coppia di chiavi RSA:** il Server genera una coppia composta da chiave pubblica e chiave privata.
2. **Invio della chiave pubblica:** il Server trasmette la chiave pubblica al Client all'inizio della comunicazione.
3. **Generazione della chiave AES:** il Client genera una chiave simmetrica che verrà utilizzata per cifrare i messaggi.

4. **Scambio sicuro della chiave:** il Client cifra la chiave AES mediante la chiave pubblica RSA ricevuta e la invia al Server.
5. **Decifratura della chiave:** il Server utilizza la propria chiave privata per recuperare la chiave AES.
6. **Comunicazione cifrata:** una volta condivisa la chiave AES, Client e Server utilizzano esclusivamente tale chiave per cifrare e decifrare i messaggi successivi.

L'applicazione di questo meccanismo consente di ridurre significativamente il rischio di esposizione dei dati durante il trasferimento.

È importante sottolineare che, in assenza del meccanismo di cifratura implementato, la comunicazione avverrebbe in chiaro e il contenuto dei messaggi potrebbe essere letto da eventuali soggetti non autorizzati in grado di intercettare il traffico. Nel sistema sviluppato, invece, anche se il traffico viene intercettato dal nodo Man-in-the-Middle, il contenuto dei messaggi risulta non interpretabile senza il possesso della chiave necessaria alla decifratura.

2.3 Tecnologie e Librerie Utilizzate

Il sistema è stato sviluppato interamente in linguaggio Python ed eseguito in ambiente locale sulla macchina personale. La scelta di Python è stata dettata dalla sua semplicità sintattica e dall'ampia disponibilità di librerie per la gestione della comunicazione di rete e della crittografia. L'esecuzione in ambiente locale, invece, ha permesso di realizzare una simulazione controllata dell'infrastruttura di rete, facilitando l'analisi del comportamento del sistema in presenza del nodo Man-in-the-Middle.

2.3.1 Libreria socket

La libreria standard `socket` è stata utilizzata per implementare la comunicazione tra Client e Server mediante protocollo TCP/IP. Essa consente la creazione e la gestione delle connessioni di rete, permettendo lo scambio di dati tra gli endpoint del sistema attraverso operazioni di invio e ricezione.

La libreria viene importata all'interno dei moduli mediante la seguente istruzione:

```
import socket
```

2.3.2 Libreria threading

La libreria standard `threading` è stata impiegata per gestire l'esecuzione concorrente del nodo Man-in-the-Middle, consentendo la gestione simultanea delle connessioni e delle operazioni di ricezione e inoltro dei dati tra Client e Server, senza bloccare il flusso principale del programma.

La libreria viene importata all'interno dei moduli mediante la seguente istruzione:

```
import threading
```

2.3.3 Modulo crypto

Per implementare il sistema di protezione delle comunicazioni è stato sviluppato un modulo dedicato denominato `crypto.py`, responsabile della gestione delle operazioni di cifratura e decifratura utilizzate durante lo scambio dei dati.

All'interno del progetto, il modulo integra funzionalità basate su crittografia asimmetrica RSA e crittografia simmetrica AES. In particolare, RSA viene utilizzato durante la fase iniziale di handshake per consentire lo scambio sicuro della chiave simmetrica, mentre AES viene impiegato successivamente per cifrare e decifrare i messaggi trasmessi tra Client e Server.

Tra le principali funzionalità implementate sono presenti la generazione delle chiavi crittografiche RSA e AES, l'importazione della chiave pubblica RSA del Server e le operazioni di cifratura e decifratura mediante entrambi gli algoritmi.

L'introduzione di questo modulo ha permesso di separare la logica crittografica dal codice dedicato alla comunicazione di rete, migliorando modularità, leggibilità e manutenibilità dell'intero sistema.

Il modulo viene importato all'interno dei diversi componenti del sistema mediante istruzioni del tipo:

```
from crypto import { }
```

2.4 Gestione degli errori e casi particolari

Durante lo sviluppo del sistema sono stati introdotti meccanismi di gestione degli errori finalizzati a garantire il corretto comportamento della comunicazione e a prevenire interruzioni inattese durante l'esecuzione:

1. **Connessione non disponibile:** nel caso in cui il Server non risulti raggiungibile oppure la connessione non possa essere instaurata correttamente, il Client interrompe l'esecuzione segnalando l'errore all'utente.
2. **Timeout di comunicazione:** per evitare situazioni di attesa indefinita durante la ricezione dei dati, il Client utilizza un meccanismo di timeout che interrompe l'attesa in assenza di risposta entro il tempo impostato.
3. **Sincronizzazione dell'handshake crittografico:** durante la fase iniziale di scambio delle chiavi, il Client attende la ricezione della chiave pubblica RSA prima di generare e trasmettere la chiave AES. La comunicazione dei dati viene avviata solamente al completamento dello scambio crittografico.
4. **Ricezione di dati non validi:** il sistema effettua controlli sui dati ricevuti per evitare errori durante le operazioni di elaborazione e decifratura.
5. **Terminazione volontaria della comunicazione:** il Client può interrompere la comunicazione mediante il comando dedicato `exit`, causando la chiusura ordinata della connessione.

La gestione di tali situazioni ha permesso di migliorare la robustezza del sistema e di rendere il comportamento dell'applicazione maggiormente prevedibile anche in presenza di condizioni anomale o interruzioni della comunicazione.

2.5 Test

Al fine di verificare il corretto funzionamento del sistema sviluppato, sono stati effettuati diversi test in ambiente locale. L'obiettivo di questa fase è analizzare il comportamento della comunicazione tra Client, Server e nodo Man-in-the-Middle in differenti scenari operativi, confrontando in particolare la trasmissione dei dati in chiaro con quella protetta mediante cifratura.

I test sono stati eseguiti simulando l'intero flusso di comunicazione e osservando direttamente gli output generati dalle varie componenti del sistema.

2.5.1 Test Comunicazione in Chiaro

In questa fase viene testata la comunicazione tra Client e Server senza l'utilizzo di meccanismi di cifratura. Il nodo Man-in-the-Middle è in grado di intercettare il traffico e visualizzare correttamente il contenuto dei messaggi scambiati tra le due entità.

```
[CLIENT] Connesso a 127.0.0.1
[CLIENT] Digita 'exit' per terminare la connessione

[CLIENT] Messaggio > Hello World!
[SERVER] ACK di risposta: Hello World!
[CLIENT] Messaggio > Questa è una comunicazione in chiaro.
[SERVER] ACK di risposta: Questa è una comunicazione in chiaro.
[CLIENT] Messaggio > Arrivederci! Chiudo la connessione....
[SERVER] ACK di risposta: Arrivederci! Chiudo la connessione....
[CLIENT] Messaggio > exit

[CLIENT] Chiusura connessione in corso
```

Figura 1: Comunicazione in chiaro da Client.

```
[MitM] In ascolto su 127.0.0.1:8000
[MitM] Client connesso da 127.0.0.1:62877
[MitM] Connessione stabilita con server 127.0.0.1:9000

[MitM C→S] [CLIENT] Messaggio: Hello World!
[MitM S→C] [SERVER] ACK di risposta: Hello World!
[MitM C→S] [CLIENT] Messaggio: Questa è una comunicazione in chiaro.
[MitM S→C] [SERVER] ACK di risposta: Questa è una comunicazione in chiaro.
[MitM C→S] [CLIENT] Messaggio: Arrivederci! Chiudo la connessione....
[MitM S→C] [SERVER] ACK di risposta: Arrivederci! Chiudo la connessione....
[MitM ERROR S→C] [WinError 10053] Connessione interrotta dal software del computer host
```

Figura 2: Comunicazione in chiaro intercettata da Man-in-the-Middle.

```
[SERVER] In ascolto su 127.0.0.1:9000
[SERVER] Connessione da 127.0.0.1:62878

[CLIENT] Messaggio ricevuto: Hello World!
[CLIENT] Messaggio ricevuto: Questa è una comunicazione in chiaro.
[CLIENT] Messaggio ricevuto: Arrivederci! Chiudo la connessione....

[SERVER] Connessione interrotta forzatamente
```

Figura 3: Comunicazione in chiaro da Server.

Il seguente test mostra il funzionamento del sistema in modalità di comunicazione in chiaro. Il nodo Man-in-the-Middle è in grado di intercettare i messaggi scambiati tra Client e Server, che risultano quindi completamente leggibili.

2.5.2 Test Comunicazione Cifrata

In questa fase viene testata la comunicazione tra Client e Server in presenza del meccanismo di cifratura. Dopo la fase iniziale di handshake, lo scambio dei dati avviene mediante algoritmo AES. Il nodo Man-in-the-Middle è in grado di intercettare il traffico, tuttavia i messaggi risultano cifrati e non interpretabili senza la chiave di decifratura.

```
[CLIENT] Connesso a 127.0.0.1
[CLIENT] In attesa della chiave pubblica RSA
[CLIENT] Chiave RSA ricevuta
[CLIENT] Comunicazione protetta stabilita
[CLIENT] Digita 'exit' per terminare la connessione

[CLIENT] Messaggio > Questa è una comunicazione protetta.
[SERVER] ACK di risposta: Questa è una comunicazione protetta.
[CLIENT] Messaggio > Solo noi possiamo comunicare correttamente!
[SERVER] ACK di risposta: Solo noi possiamo comunicare correttamente!
[CLIENT] Messaggio > Alla prossima!
[SERVER] ACK di risposta: Alla prossima!
[CLIENT] Messaggio > exit

[CLIENT] Chiusura connessione in corso
```

Figura 4: Comunicazione cifrata da Client.

```
[MitM] In ascolto su 127.0.0.1:8000
[MitM] Client connesso da 127.0.0.1:63583
[MitM] Connessione stabilita con server 127.0.0.1:9000

[MitM S->C] [SERVER] ACK di risposta: RSA-PUB:-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAOxjHUIz+UV3YAD7FueJe
bbr3EtzuGdznNgi5zSKyrr+th1mBt5LzocnaGhH7a/c5y0S28yqV6G+pP/0331p4
JwmEjgsPdF0j3KkF8xCXzySebYlZy516yI+uL0oTjN1HLe1pGCwzGGTKTZPzNCw
wsZi/qRaAGO+XpUungY8jndfUhwQDa3WkmeOg94TJ5XrzMQcP3iErytLyKrud8x
FDg+Iv2BQwdwjBwZLwrWEnSMzmTKUM5rjiye5xHS4xJLgha/epeYoh6iqIL2TQGe
3lgYrkXV9Im6pVQZ+XsRkJih4KTzw+AKwKvGSt00Cfiyi7PZZedQvqqMfnzGj/L
iQIDAQAB
-----END PUBLIC KEY-----
[MitM C->S] <BIN DATA>
[MitM C->S] [CLIENT] Messaggio: bfrGjcTvPRGcnG4y85VzX6X0jY+BuM+mroDNA/PvR/E0w2HLwDfKGGvmiSXfH02dfP+5AoG0cTP4RzUfIWST/A==
[MitM S->C] [SERVER] ACK di risposta: njNJGvbU0pfLyUC8wSmkm9M2EMe0ViI5CR+N8Fr0DoGUAHjK630L33jup6JVV2fXEGkMtY92mBqu5aRYErBmDw==
[MitM C->S] [CLIENT] Messaggio: E7TGwsvr+XYgsfwy3qIrsRBYuqzNI/Vk6evRiCKP95Agd4DNlUV7sd8n3dxMRzwua4I54uYUNq19fvZFsUbRQ==
[MitM S->C] [SERVER] ACK di risposta: G1SmzYAD2aoXMRa5Pzb00b1aPH0qeRDYj0zqxNeAXyctPAdzYoUFiCDreJD44gcEX9vAfbivthQuatrC0TnmDg==
[MitM C->S] [CLIENT] Messaggio: Q6TLBTch80aCA+DDLUpP/D3o8Q5bpXYS0kQG9IzQtTc=
[MitM S->C] [SERVER] ACK di risposta: lZxTvV58ApAIUY6bc1qKQUMV5tZ14QdJsIHAFqWxc8A=
[MitM ERROR S->C] [WinError 10053] Connessione interrotta dal software del computer host
```

Figura 5: Comunicazione cifrata intercettata da Man-in-the-Middle.

```
[SERVER] In ascolto su 127.0.0.1:9000
[SERVER] Connessione da 127.0.0.1:63584
[SERVER] Chiave AES ricevuta e decifrata
[SERVER] Inizio comunicazione protetta

[CLIENT] Messaggio ricevuto: Questa è una comunicazione protetta.
[CLIENT] Messaggio ricevuto: Solo noi possiamo comunicare correttamente!
[CLIENT] Messaggio ricevuto: Alla prossima!

[SERVER] Connessione interrotta forzatamente

[SERVER] Connessione chiusa
```

Figura 6: Comunicazione cifrata da Server.

Il test permette di evidenziare come l'utilizzo della crittografia garantisca la riservatezza delle informazioni trasmesse, impedendo la lettura del contenuto da parte di nodi intermedi non autorizzati.

2.5.3 Test Gestione Errori

In questa sezione vengono analizzati alcuni casi particolari legati alla gestione delle anomalie durante la comunicazione tra Client, Server e nodo Man-in-the-Middle.

Chiusura intenzionale della connessione da parte del Server: in questo test viene simulata l'interruzione improvvisa del Server durante la comunicazione con il Client, al fine di verificare la gestione della disconnessione.

```
[CLIENT] Connesso a 127.0.0.1
[CLIENT] In attesa della chiave pubblica RSA
[CLIENT] Chiave RSA ricevuta
[CLIENT] Comunicazione protetta stabilita
[CLIENT] Digita 'exit' per terminare la connessione

[CLIENT] Messaggio > Controllo status server.
[SERVER] ACK di risposta: Controllo status server.
[CLIENT] Messaggio > Controllo 2 status server

[CLIENT] Connessione interrotta da parte del server
```

Figura 7: Gestione della chiusura improvvisa del Server

Dal test emerge che la chiusura improvvisa del Server comporta l'interruzione della comunicazione da parte del Client, che rileva la perdita della connessione e termina l'esecuzione in modo controllato.

Timeout di comunicazione: viene verificato il comportamento del Client nel caso in cui la risposta del Server non venga ricevuta entro il tempo prestabilito.

```
[CLIENT] Connesso a 127.0.0.1
[CLIENT] In attesa della chiave pubblica RSA
[CLIENT] Chiave RSA ricevuta
[CLIENT] Comunicazione protetta stabilita
[CLIENT] Digita 'exit' per terminare la connessione

[CLIENT] Messaggio > Ciao server, come stai?

[CLIENT] Timeout raggiunto: nessuna ACK ricevuta
[CLIENT] Messaggio > Rispondi!!!
[SERVER] ACK di risposta: Ciao server, come stai?
[CLIENT] Messaggio > exit

[CLIENT] Chiusura connessione in corso
```

Figura 8: Gestione del timeout del Client

Dal test emerge che, in caso di ritardo nella risposta del Server, il Client segnala correttamente il raggiungimento del timeout. Tuttavia, la risposta del Server non viene persa, ma viene comunque elaborata e inviata successivamente. Questo può generare uno sfasamento tra richiesta e risposta nelle comunicazioni successive.

Handshake non completato: viene simulata un'interruzione durante la fase iniziale di scambio delle chiavi crittografiche, verificando che la comunicazione non venga avviata senza il completamento della procedura.

```
[CLIENT] Connesso a 127.0.0.1
[CLIENT] In attesa della chiave pubblica RSA

[CLIENT] Timeout raggiunto: Handshake non completato!
```

Figura 9: Gestione handshake non completata del Client

Dal test si osserva che, in assenza della ricezione della chiave pubblica RSA, il Client non procede con la generazione della chiave AES.

3 Considerazioni Finali

In questa sezione vengono riportate alcune considerazioni conclusive sul progetto sviluppato, evidenziando i risultati ottenuti, i principali aspetti emersi durante l'implementazione e alcune possibili evoluzioni future del sistema. Lo sviluppo dell'applicazione ha permesso di analizzare in modo pratico il funzionamento delle comunicazioni su rete TCP/IP e di osservare gli effetti che l'introduzione di un meccanismo di cifratura produce sulla sicurezza della comunicazione.

3.1 Risultati Ottenuti

L'obiettivo iniziale del progetto era realizzare una simulazione in linguaggio Python di una comunicazione Client/Server con nodo Man-in-the-Middle su rete TCP/IP, con particolare attenzione agli aspetti legati alla sicurezza della trasmissione dei dati.

Attraverso l'implementazione dei moduli Client, Server e Man-in-the-Middle è stato possibile riprodurre uno scenario di comunicazione rappresentativo, consentendo sia lo scambio dei messaggi tra le due entità sia l'intercettazione del traffico da parte di un soggetto terzo non autorizzato.

I test effettuati hanno evidenziato le differenze tra una comunicazione in chiaro e una comunicazione protetta mediante meccanismi di cifratura. In particolare, nella modalità non protetta il nodo Man-in-the-Middle è risultato in grado di leggere direttamente il contenuto dei messaggi intercettati, mentre nella modalità cifrata il traffico è rimasto intercettabile ma non interpretabile senza il possesso della chiave di decifratura.

Il progetto ha inoltre permesso di approfondire in modo pratico il funzionamento delle socket TCP/IP, il processo di instaurazione della comunicazione tra endpoint e l'utilizzo combinato di crittografia asimmetrica RSA e simmetrica AES per la protezione dello scambio dei dati.

In conclusione, gli obiettivi prefissati possono considerarsi raggiunti e il progetto ha consentito di verificare concretamente i concetti teorici studiati.

3.2 Limiti del Sistema Implementato

Nonostante il corretto funzionamento del sistema e il raggiungimento degli obiettivi prefissati, il progetto presenta alcune limitazioni derivanti dalle scelte progettuali adottate e dalla natura didattica dell'elaborato.

In primo luogo, l'intero sistema è stato sviluppato ed eseguito in ambiente locale, senza il coinvolgimento di una rete distribuita reale. Questa scelta ha permesso di semplificare la configurazione e concentrare l'attenzione sul comportamento della comunicazione e sugli aspetti di sicurezza, ma ha limitato la possibilità di valutare il sistema in condizioni di rete realistiche.

Nel contesto reale, un attacco Man-in-the-Middle consiste nell'inserimento di un nodo terzo all'interno di una comunicazione già stabilita, con l'obiettivo di intercettare e, eventualmente, alterare i dati trasmessi senza che le parti coinvolte ne siano consapevoli.

Questo inserimento può avvenire attraverso diverse tecniche a livello di rete, come il reindirizzamento del traffico mediante ARP spoofing in reti locali oppure la creazione di un punto di accesso non autorizzato che si interpone tra le due entità comunicanti.

Nel sistema implementato, tale comportamento è stato simulato introducendo un nodo intermedio che si colloca logicamente tra Client e Server, ricevendo i pacchetti in transito e inoltrandoli successivamente al destinatario, consentendo così l'analisi e l'eventuale modifica del contenuto dei messaggi, pur non riproducendo completamente le dinamiche di un attacco su rete reale.

3.3 Possibili Miglioramenti Futuri

In questa sezione vengono descritti i possibili miglioramenti futuri applicabili al sistema, ponendo soprattutto attenzione agli aspetti legati alla sicurezza e certificazione degli host che comunicano e sulla gestione di scambio dei dati.

In particolare, una possibile miglioramento riguarda l'introduzione di un controllo sull'identità e validità degli host comunicanti, con lo scopo di ridurre maggiormente il rischio di intercettazioni e impersonificazioni.

Un ulteriore miglioramento potrebbe riguardare il sistema di cifratura dei messaggi, integrando in modo più strutturato i meccanismi basati su RSA e AES già utilizzati nel progetto, così da ottenere una gestione più sicura dello scambio delle chiavi e una maggiore protezione dei dati trasmessi, aumentando complessivamente sicurezza e integrità della comunicazione.

3.4 Considerazioni Personali

In questa ultima sezione vengono riportate alcune considerazioni personali maturate durante lo sviluppo del progetto.

Come evidenziato nell'introduzione, la realizzazione di questo progetto ha permesso di approfondire in modo pratico le dinamiche della comunicazione TCP/IP e di comprendere come un soggetto terzo non autorizzato possa intercettare, analizzare e, in determinate condizioni, alterare informazioni trasmesse in assenza di adeguati meccanismi di sicurezza.

Inoltre, a seguito dello sviluppo di tutti e tre i moduli, si può osservare come la scelta di implementare il sistema in Python sia stata adeguata, grazie alla sua scalabilità, dinamicità e alla ampia disponibilità di librerie standard che hanno permesso di sviluppare tutte le funzionalità previste.

La scelta di eseguire l'intero sistema in ambiente locale si è rivelata particolarmente efficace, in quanto ha permesso di sviluppare e testare le diverse componenti in modo stabile e controllato, facilitando la fase di implementazione e la verifica del corretto funzionamento della comunicazione.

Bibliografia

- [1] Python Software Foundation, libreria: *socket* — *Low-level networking interface*
<https://docs.python.org/3/library/socket.html>
- [2] Python Software Foundation, libreria: *threading* — *Thread-based parallelism*,
<https://docs.python.org/3/library/threading.html>
- [3] PyCryptodome, *Examples: RSA and AES encryption in Python*,
<https://pycryptodome.readthedocs.io/en/latest/src/examples.html>
- [4] A. Della Selva, *Dispense e materiale didattico del corso di Reti di Calcolatori*, Corso di Informatica Applicata, Università degli Studi di Urbino Carlo Bo.